

TRIBHUWAN UNIVERSITY

CENTRAL DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



Advanced Operating Systems Assignment No-1

Submitted By:

Name: Yujan Basnet

Roll No: 18

Msc, CSIT

Submitted to:

Prof. Dr. Binod Kumar Adhikari
CDCSIT, TU

Table Of Contents

Inter Process Communication (IPC).....	2
Introduction	2
Critical Section and Critical Section Problem.....	3
Introduction	3
Conditions for a solution	3
Different ways of solving the CSP are:	3
Conclusion.....	4
Semaphores	5
Introduction	5
Operating System Structure and Synchronization Mechanisms.....	5
Performance Evaluation	5
Reliability and Debugging.....	6
Challenges Faced During Development.....	6
Conclusion.....	6
Monitors	7
Introduction	7
Pseudo Code.....	7
Conclusion.....	8
Peterson's Algorithm.....	9
Introduction	9
Conditions for a Valid Critical Section Solution.....	9
Two-Process Solution (Algorithm 1)	9
N-Process Solution (Algorithm 2 - Tournament Algorithm)	9
Conclusion.....	9
References	11

Inter Process Communication (IPC)

Introduction

Inter-Process Communication (IPC) is a set of techniques and mechanisms that enable different processes to exchange data, share information, and coordinate their activities within an operating system.

Process Synchronization occurs whenever two or more process coordinate their activities based on shared conditions. But the main problem that arises during this is race condition where two or more than two resources wait (for reading or writing) for the same shared resource. The direct communication can be enabled by Inter Process Communication (IPC) between different process using message passing. [1]

There can be many synchronization problems like mutual exclusion, producer-consumer, and readers-writer's problems. These problems can lead into challenges like busy waiting, starvation, deadlock. IPC mechanisms that can be used are Message passing, pipes, and software interrupts. [1]

There are many Inter Process Communication (IPC) mechanisms like:

a. Pipes

In pipes there is unidirectional communication meaning data flows in only one direction (writer to reader). They are used between a parent and child process or threads of the same process. They are synchronized automatically when the buffer is full the writer waits and when empty the reader waits. The major limitations in this are that it only works in related processes and due to the fixed buffer size, there are many delays. [2]

b. Sockets

In sockets the communication is bidirectional. The major use is between the communication between process on that same machine or the network. This uses the TCP/IP protocol to split data into packets. The major limitations are that sockets have higher latency compared to pipes and shared memory because of packet-based transmission. [2]

c. Shared Memory

This is the fastest IPC mechanism where the process accesses a common memory region directly. Due to the lack of need system call or data copying, it is highly efficient. The major limitations of shared memory are due to the lack of synchronization process need extra mechanisms like semaphores to manage access and security risk can occur if the process access the shared memory incorrectly. [2]

Critical Section and Critical Section Problem

Introduction

Critical section refers to a segment of code where multiple processes or thread access shared resources.

When two or more processes try to modify shared data at the simultaneously the race condition can occur which is called critical section problem.

Conditions for a solution

1. Mutual Exclusion

Only one process enter can enter the critical section at a time to prevent simultaneous modification of shared data.

2. Progress

When critical section doesnot have any process then a new process should be allowed to entered.

3. Bounded Waiting

To prevent starvation each process should have a time limit of how much it should wait access to the critical section. [3]

Different ways of solving the CSP are:

1. Software Solutions

a. Disabling Interrupts

All interrupts are disabled when a process enters the critical section but it only works in single-processor system.

b. Dekker's Algorithm & Peterson's Algorithm

It uses flags and turn-taking to enforce mutual exclusion. More efficient than disabling interrupts but still has performance overhead.

c. Spinlocks (Test & Set Instruction)

A process continuously checks if the resource is available. The problems are that it wastes CPU cycles while waiting.

d. Semaphores

A semaphore is a special variable that controls access to shared resources. They are efficient because it blocks processes instead of making them wait actively. [3]

2. Hardware Solutions

a. Atomic Instructions (Test-and-Set, Compare-and-Swap)

These ensure that only one process modifies shared memory at a time. They are used in modern multi-core processors.

b. Operating System Support

OS provides built-in functions like:

- Windows APIs: InitializeCriticalSection(), EnterCriticalSection(), and LeaveCriticalSection().
- Linux Synchronization Primitives: pthread_mutex_lock(), pthread_mutex_unlock().

The major insights can be there is no perfect solution exists yet that meets all CSP conditions without trade-offs. Spinlocks and busy-waiting methods are inefficient because they waste CPU cycles. Semaphore-based solutions are the most effective because they block waiting processes instead of making them check continuously. Windows XP & Modern OS Implementations uses a combination of critical sections, semaphores, and atomic operations to ensure safe resource sharing. [3]

Conclusion

Critical Section Problem remains an open challenge in concurrent computing. Software solutions (like Peterson's Algorithm) help, but hardware-based approaches (like semaphores) are more efficient. Modern operating systems use a hybrid approach, combining hardware and software techniques for maximum efficiency.

Semaphores

Introduction

A semaphore is a synchronization mechanism used in concurrent programming to control access to shared resources by multiple processes or threads.

Edsger Dijkstra introduced semaphores in 1965 as a powerful mechanism for process synchronization. There are two primary types [4]:

- Binary Semaphores: Can have only two values (0 or 1)
- Counting Semaphores: Can have an arbitrary number of values

Operating systems play a crucial role in managing concurrent processes and ensuring efficient execution of multiple tasks. [5] presents Boss 2, an advanced multiprogramming operating system developed for the RC 4000 computer. Boss 2 takes an asynchronous approach using semaphores and message buffers for process synchronization.

The system was designed to handle: [4]

- Batch jobs
- Remote job entry
- Time-sharing (interactive jobs)
- Process control jobs
- Job-to-job communication

A key design feature of Boss 2 was its ability to prevent deadlocks while maintaining system efficiency through dynamic process scheduling and prioritization.

Operating System Structure and Synchronization Mechanisms

Boss 2 used semaphores and message-based synchronization to manage parallel processes. Each process had its own message queue, enabling safe communication without causing deadlocks. The system supported job scheduling, disk management, and process prioritization dynamically. [5]

Performance Evaluation

[5] reports CPU utilization of 40-50% for jobs and 10-20% for system overhead. Boss 2 was tested with 20 terminals, 64K words of core storage, and a small drum. The response time for simple commands was less than 0.5 seconds, demonstrating efficiency.

Reliability and Debugging

Boss 2 was highly reliable, often running for weeks without crashes. Errors in the system were mainly programming mistakes (e.g., improper resource handling). Developers used test output logs to debug issues, ensuring system stability.

Challenges Faced During Development

Due to the underestimation of instructions, the required instructions were three times higher than expected. Due to interface complexity, The Monitor (nucleus) needed multiple modifications to support Boss 2. Due to resource management issues, Early versions suffered from thrashing and scheduling inefficiencies.

Conclusion

Boss 2 was an innovative operating system that successfully demonstrated semaphore-based synchronization for managing concurrent processes. It provided efficient job scheduling, deadlock prevention, and high reliability on the RC 4000 system. Despite some challenges in estimation and debugging, the system proved to be stable, efficient, and adaptable. The insights from [5] contributed to later multiprogramming and time-sharing systems in modern computing.

Monitors

Introduction

Monitor is a synchronization construct used to manage access to shared resources in a concurrent or multithreaded environment. It helps prevent race conditions by ensuring that only one thread or process can access a critical section of code or resource at a time.

A monitor is a module that encapsulates data and the procedures that operate on it, ensuring mutual exclusion and synchronization among processes. This concept is particularly useful for resource allocation and scheduling.

The primary goal of an operating system is to manage shared resources among multiple programs. A monitor consists of local administrative data and procedures, which can only be accessed in a controlled manner. Only one process can execute a monitor procedure at a time, preventing concurrent modifications that could lead to inconsistencies. Wait and signal operations are used to handle process synchronization. [6]

Some practical examples of monitor are [6]:

- **Bounded Buffer:** Implements producer-consumer synchronization.
- **Alarm Clock:** Delays processes for a specified time.
- **Disk Head Scheduler:** Optimizes disk movement using the elevator algorithm.
- **Readers-Writers Problem:** Ensures fair access control, preventing indefinite writer starvation.

Pseudo Code

```
Monitor monitorName{
    variables_declaration;
    condition_variables;

    procedure p1{ ... };
    procedure p2{ ... };
    ...
    procedure pn{ ... };
    {
        initializing_code;
    }
}
```


Conclusion

Monitors provide a structured, efficient, and provably correct way to manage concurrency in operating systems. They offer a safer alternative to semaphores, making it easier to design and verify synchronization mechanisms. However, their practical implementation requires careful scheduling strategies to avoid inefficiencies and deadlocks. [6]

Peterson's Algorithm

Introduction

Peterson's Algorithm is used to handle critical section problem. It is designed specifically for two processes that need to share a resource safely. Peterson's algorithm uses two shared variables to coordinate:

- flag: This is like a "want to use" signal. Each process has its own flag.
- turn: This says whose turn it is to use the resource.

Peterson's Algorithm focuses on solving the critical section problem, where multiple processes must coordinate to ensure mutual exclusion. [7]

Conditions for a Valid Critical Section Solution

- No two processes can be in their critical sections simultaneously (Mutual Exclusion).
- If a process wants to enter its critical section, it should eventually be allowed to do so, even with process failures (No Deadlock).

Two-Process Solution (Algorithm 1)

- Uses three states per process: "~" (idle), "T", and "F".
- Process P0 waits for P1's state to change before entering its critical section.
- Process P1 follows a complementary rule to ensure fairness.
- The solution guarantees mutual exclusion and prevents deadlock.
- It is proven that a two-state solution is impossible because it would lead to indefinite lockout.

N-Process Solution (Algorithm 2 - Tournament Algorithm)

- Extends the two-process solution using a divide-and-conquer approach.
- Processes compete in groups of two until a single winner emerges.
- Requires $O(\log n)$ states, which is better than previous $O(n)$ solutions.
- Guarantees mutual exclusion and prevents lockout.

Conclusion

[7] presents efficient solutions to the critical section problem in distributed systems, improving message size, fairness, and execution time. The two-process solution (Algorithm 1) ensures mutual exclusion using three states, proving a two-state solution is impossible. The Tournament Algorithm (Algorithm 2) extends this to n processes with $O(\log n)$ states,

significantly reducing complexity. The tournament approach achieves $O(n^2)$ time complexity, outperforming previous $O(2^n)$ solutions.

References

- [1] C. E. WILLS, "Process Synchronization and IPC," *ACM Computing Surveys*, vol. 28, no. 1, pp. 209-211, 1996.
- [2] K. K. J. Aditya Venkataraman, "Evaluation of Inter-Process Communication Mechanisms," *University of Wisconsin-Madison*.
- [3] E. A. G. E. A. M. Er.Ankit Gupta, "RESEARCH PAPER ON SOFTWARE SOLUTION OF CRITICAL SECTION PROBLEM".
- [4] E. Dijkstra, "Co-operating sequential processes," in *Programming languages : NATO Advanced Study Institute : lectures given at a three weeks Summer School held in Villard-le-Lans, 1966 / ed. by F. Genuys*, 1968.
- [5] N. B. B. C. Soren Lauesen, "A Large Semaphore Based Operating System," *Communication of the ACM*, vol. 18, no. 7, 1975.
- [6] C. Hoare, "Monitors: An Operating System Structuring Concept," *The Communication of ACM*, vol. 17, no. 10, 1974.
- [7] G. L. P. a. M. J. Fischer, "ECONOMICAL SOLUTIONS FOR THE CRITICAL SECTION PROBLEM IN A DISTRIBUTED SYSTEM," *Department of Computer Science, University of Washington*.